

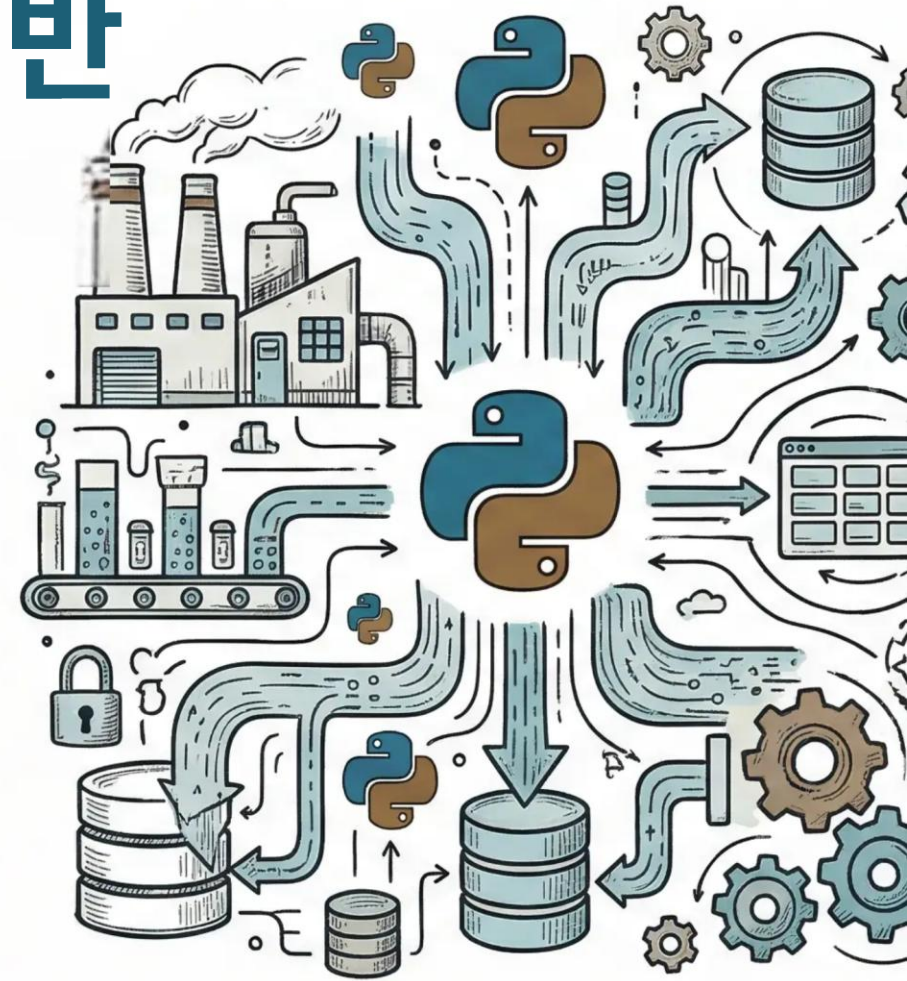
[기술공통] 데이터 엔지니어링



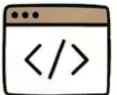
제조 AX를 위한 Python 기반 데이터 엔지니어링 실무

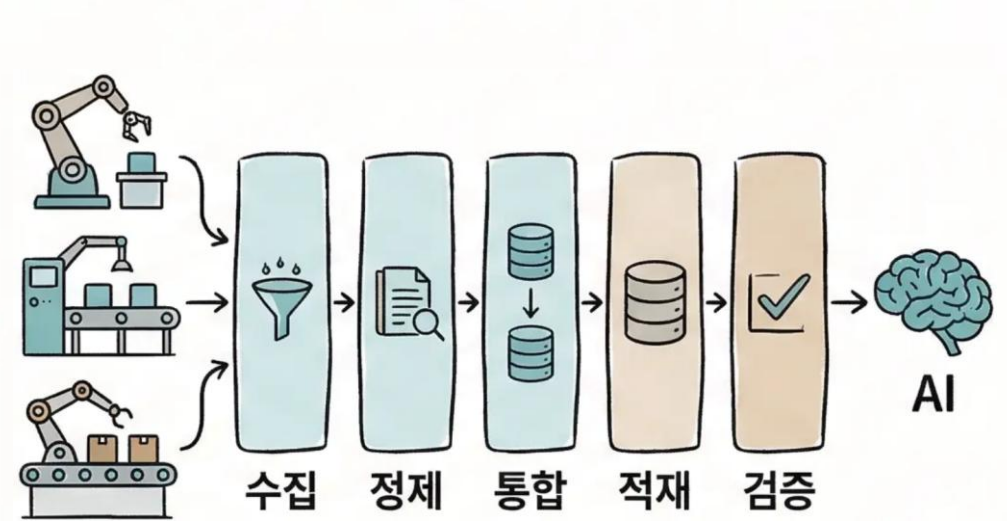
**pandas · 데이터 전처리 · 데이터 수집 ·
SQLite 적재 · ETL · 품질 점검**

3주간의 실무 중심 과정으로 제조 현장의 데이터를
Python으로 처리하고 활용하는 방법을 학습합니다



제조 AX와 데이터 엔지니어링 실무

- 제조 AX의 출발점은 데이터입니다  
 - AI 모델보다 먼저 필요한 것은 정리된 데이터
 - 데이터 품질이 AI 성과를 결정합니다
- 모델보다 먼저 데이터 파이프라인  
 - 수집 → 정제 → 통합 → 적재 → 검증
 - 지속 가능한 AI 시스템의 기반
- 실습 중심의 실무형 접근  
 - 실제 제조 데이터로 전체 흐름 경험
 - 코드와 결과물로 배우는 데이터 엔지니어링



데이터 없이는 AI도 없습니다.

이번 과정은 AI 모델링 이전에 반드시 필요한 데이터 흐름을 다루는 실무 중심 수업입니다.

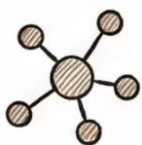
이번 과정에서 무엇을 배우는가



- 제조 데이터를 읽고 구조를 이해합니다
- AI4I 2020 데이터셋으로 실제 제조 데이터 경험
- DataFrame 구조와 행-열 관계 파악



- pandas로 데이터를 정리합니다
- 결측치, 중복, 이상치 처리
- 컬럼 표준화와 파생 변수 생성



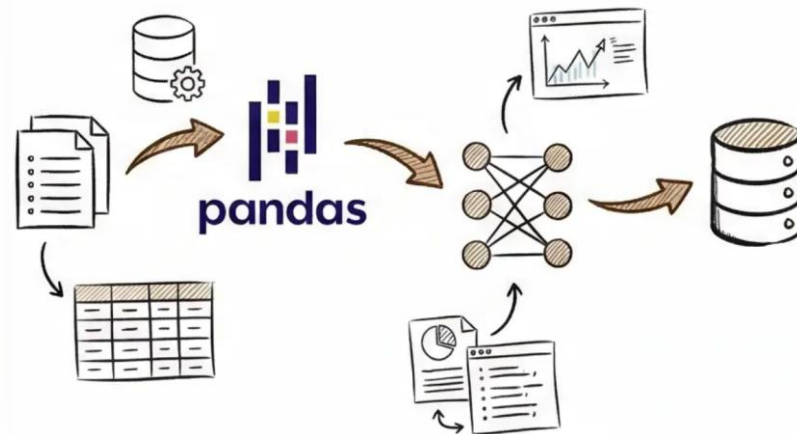
- 여러 출처의 데이터를 연결합니다
- CSV, JSON, API 데이터 통합
- merge를 통한 데이터 결합 기법



- SQLite에 적재하고 품질 점검까지 경험합니다
- ETL 파이프라인 구축
- 데이터 검증과 로그 관리

실무 중심 학습

“이론보다는 실제 제조 현장에서 마주할 수 있는 데이터로 직접 실습하며, 바로 적용 가능한 기술을 습득합니다”



3주 전체 로드맵

제조 데이터 엔지니어링의 단계별 학습 여정

1 1주차

데이터 이해와 전처리

- AI4I 2020 데이터셋 분석
- Pandas 기반 EDA 수행
- 데이터 정제 및 저장

 ai4i_cleaned.csv

2 2주차

다양한 소스 통합

- 여러 CSV 파일 결합
- JSON 데이터 처리
- 데이터 소스 통합

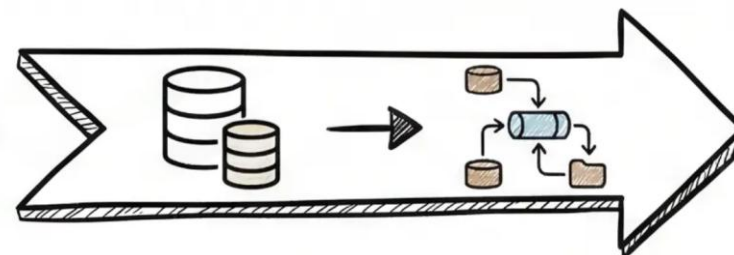
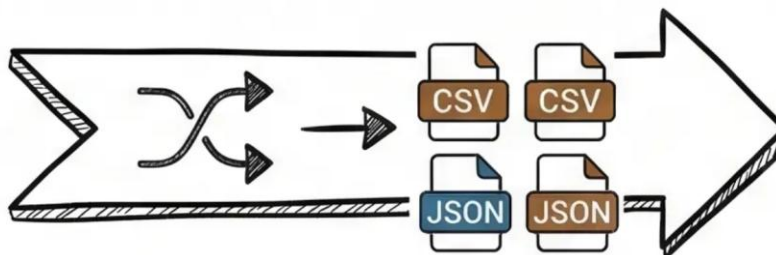
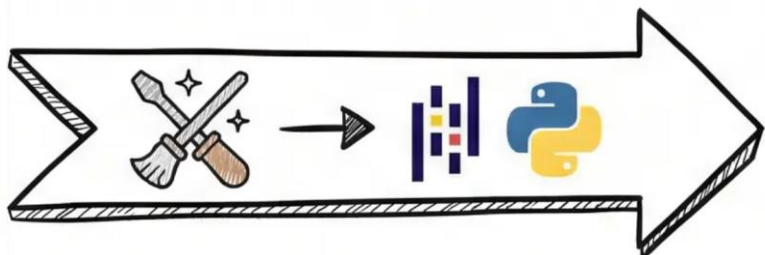
 ai4i_enriched.csv

3 3주차

SQLite 적재와 ETL

- 데이터베이스 적재
- ETL 파이프라인 구축
- 품질 점검 및 로깅

 manufacturing_pipeline.db



1주차 학습 목표

AI4I 데이터 구조를 이해한다

실제 제조 데이터셋의 행과 열 구조 파악

DataFrame으로 데이터를 점검한다

pandas 기본 함수로 데이터 상태 확인

기초 EDA와 전처리를 수행한다

데이터 품질 점검과 정제 작업 실습

정제 결과를 저장한다

ai4i_cleaned.csv 파일로 결과물 생성



오늘 만들 결과물

1주차 실습을 통해 완성할 구체적인 산출물들



원본 제조 데이터 읽기

AI4I 2020 데이터셋 로드 및 구조 파악



데이터 상태 점검

결측치, 중복, 자료형 검증 리포트



기초 시각화

분포 확인 차트 및 이상치 탐지 그래프



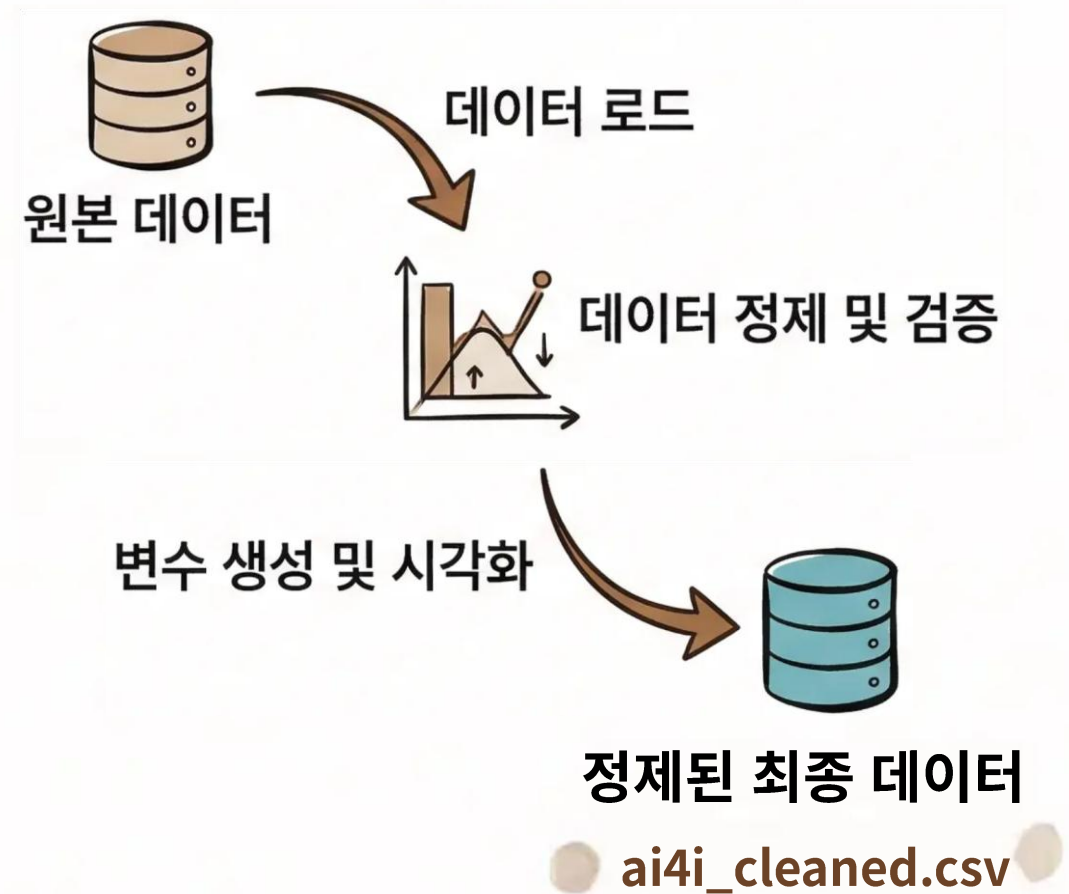
파생 변수와 이상치 플래그 생성

온도 차이, 마모 구간 등 새로운 변수 추가

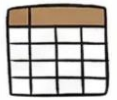


최종 결과물: ai4i_cleaned.csv

정제 완료된 데이터셋 파일



왜 pandas가 중요한가



실무 데이터는 대부분 표 형태다

- 센서 로그, 품질 데이터, 생산 실적 모두 행과 열 구조



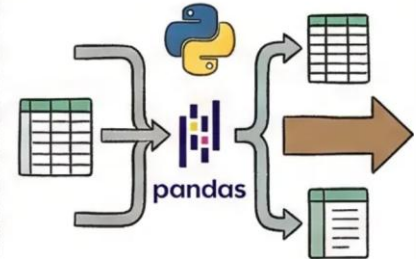
엑셀 반복 작업을 자동화할 수 있다

- 매일 반복하는 데이터 정리 작업을 코드로 처리



대용량 데이터도 빠르게 처리한다

- 엑셀로 열기 어려운 큰 파일도 쉽게 다룸
- 수만 행의 데이터도 몇 초 만에 분석 가능



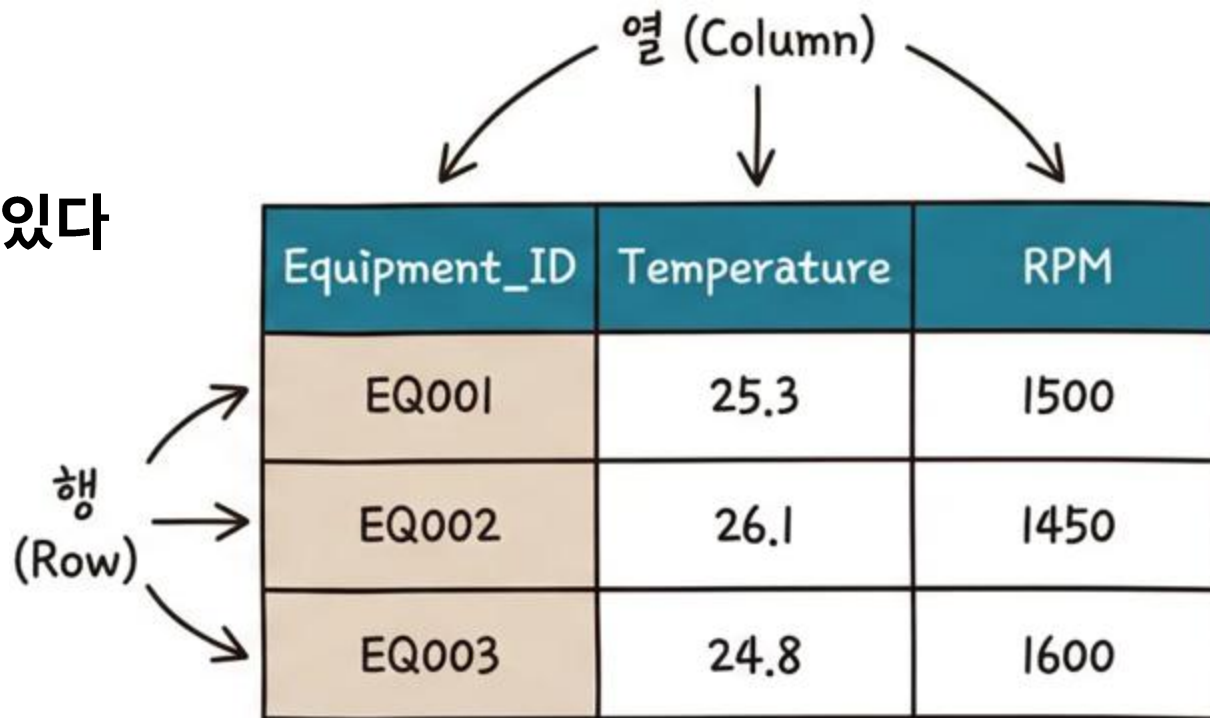
DataFrame이란 무엇인가

행과 열로 이루어진 표 구조

엑셀과 비슷하지만 코드로 반복 처리할 수 있다

가장 자주 다루는 실무 데이터 형태

- 2차원 테이블 형태의 데이터 구조
- 행(Row): 개별 기록이나 관측치
- 열(Column): 데이터의 속성이나 특징
- 코드 기반 자동화로 대량 데이터 처리 가능

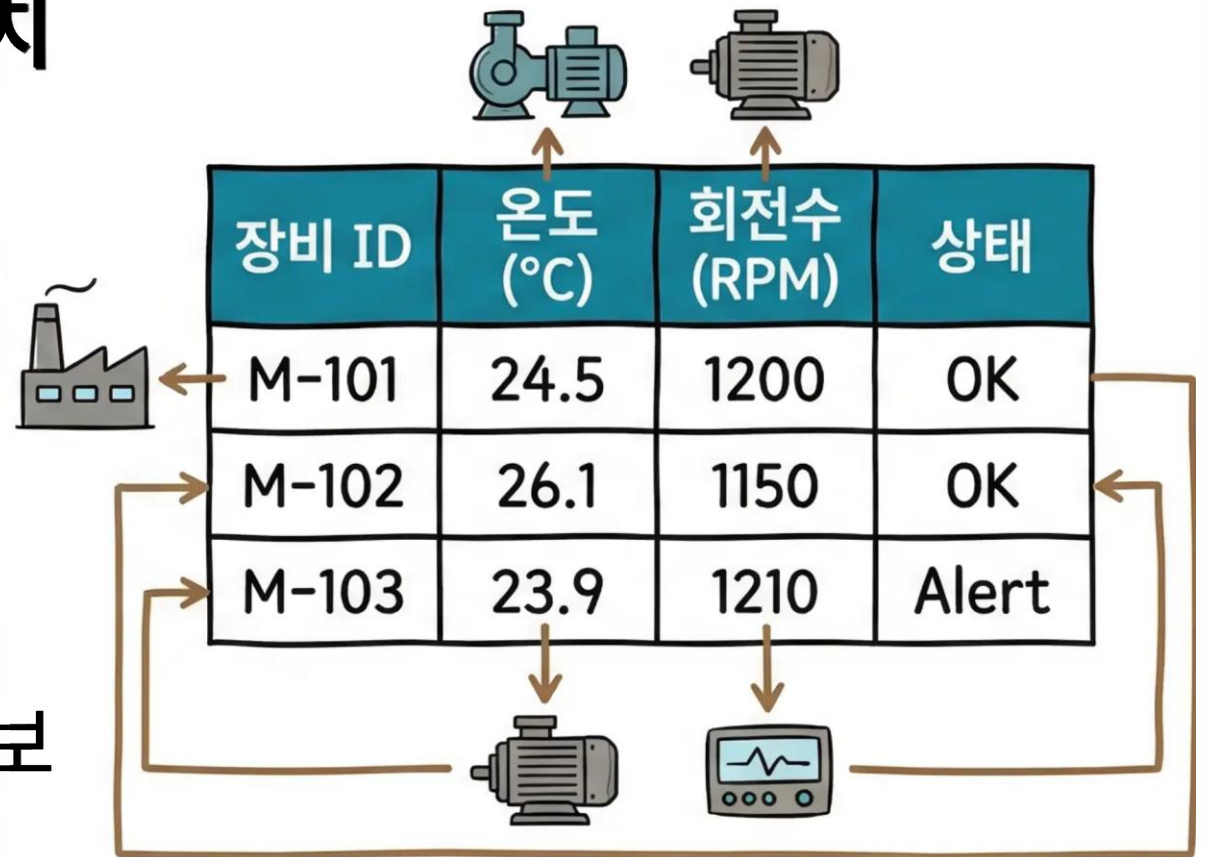


The diagram illustrates a DataFrame as a table with three columns and three rows. The columns are labeled 'Equipment_ID', 'Temperature', and 'RPM'. The rows are labeled 'EQ001', 'EQ002', and 'EQ003'. Arrows indicate the structure: a curved arrow labeled '열 (Column)' points to the column headers, and a curved arrow labeled '행 (Row)' points to the row headers.

Equipment_ID	Temperature	RPM
EQ001	25.3	1500
EQ002	26.1	1450
EQ003	24.8	1600

행과 열로 보는 데이터 구조

- **행(Row):** 하나의 기록 또는 관측치
 - 한 대의 장비 상태
 - 특정 시점의 측정값
 - 개별 제품의 품질 정보
- **열(Column):** 기록의 특정 속성
 - 온도, 압력, 회전수 등 센서값
 - 장비 ID, 제품 타입 등 식별정보
 - 상태, 등급 등 분류정보



“데이터 해석은 행과 열의 의미를 이해하는 것에서 시작됩니다”

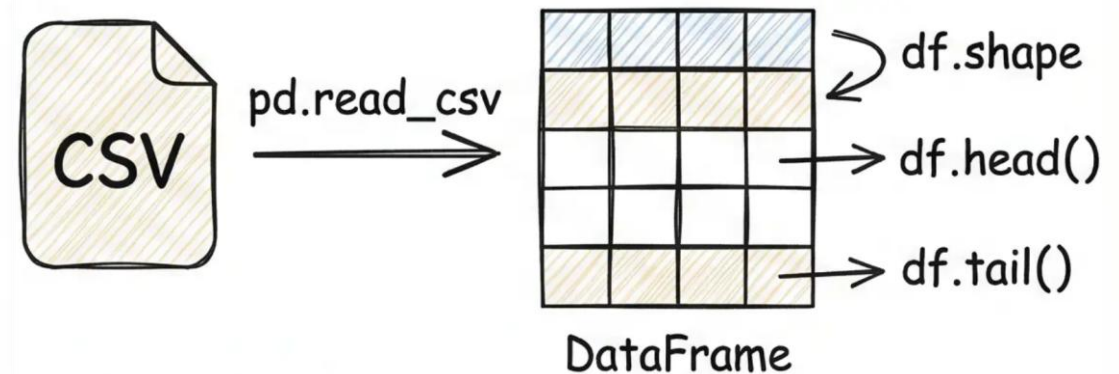
CSV 읽기와 기본 확인

핵심 함수들

- `pd.read_csv('파일명.csv')`
 - CSV 파일을 DataFrame으로 읽기
- `df.shape` - 데이터의 행과 열 개수 확인
- `df.head(5)` - 상위 5개 행 미리보기
- `df.tail(5)` - 하위 5개 행 미리보기

실습 포인트

- 파일 경로가 정확한지 먼저 확인
- 데이터 크기를 `shape`로 파악
- `head()`로 컬럼명과 데이터 형태 확인
- `tail()`로 데이터 끝부분 상태 점검



💡실무 팁: 큰 데이터셋은 `head()`로 구조를 먼저 파악한 후 전체 작업을 진행하세요

info()와 describe()로 데이터 이해하기

데이터 작업의 첫 번째 단계

데이터 작업의 첫 번째 단계

info() 함수의 역할

- 각 컬럼의 데이터 타입 확인
- Non-Null 값의 개수 파악
- 전체 메모리 사용량 정보
- 결측치 여부를 한눈에 확인

describe() 함수의 역할

- 수치형 컬럼의 기술통계량
- 평균, 표준편차, 최소/최대값
- 사분위수로 분포 파악
- 데이터 범위와 중심 경향성 이해

	Dtype
id	objet64
lib	objet64
atz	objet64
...	7 (Na)

	non-lung	statistic
waae	177.014	76.9000
max.aege	10.454	0.0000
min.stock	23.209	0.0000
max.stock	30.900	0.0000
max.stock	30.000	0.0000
dtype: summary		

실무에서 활용하는 방법

데이터 품질 체크리스트

- ✓ 자료형이 올바른가?
- ✓ 결측치는 얼마나 있는가?
- ✓ 수치 범위가 합리적인가?
- ✓ 이상치가 보이는가?

분석 전 필수 점검사항

- 컬럼별 완전성 확인
- 예상 범위 내 값 검증
- 데이터 분포 특성 파악

AI4I 2020 데이터셋 소개

⚙ 데이터셋 개요

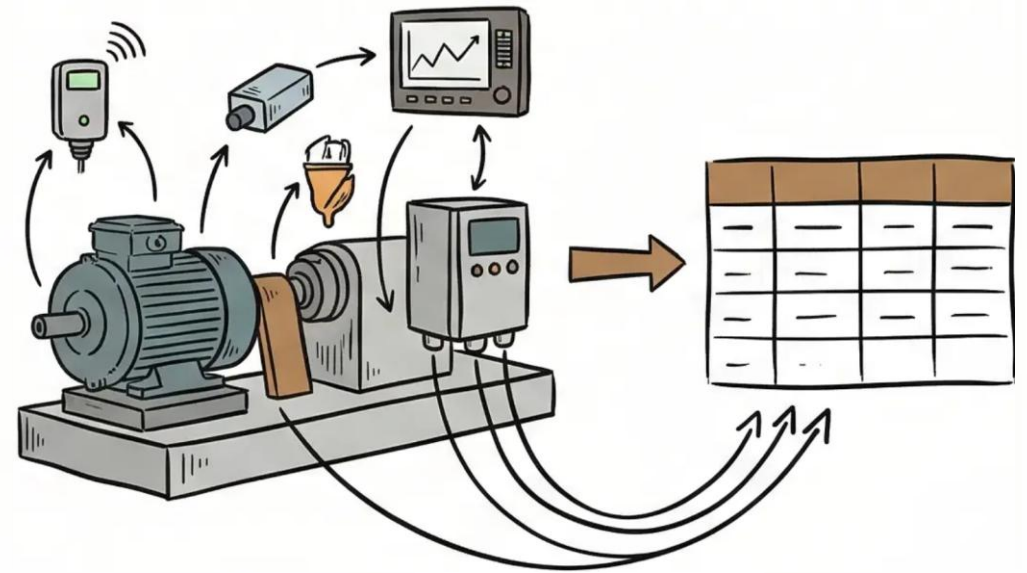
- 예지보전을 위한 실제 제조 환경 반영
- 10,000개 데이터 포인트와 14개 특징으로 구성
- 머신러닝 학습에 최적화된 구조

📡 포함된 정보 유형

- 센서 데이터: 온도, 회전속도, 토크 측정값
- 운전 변수: 공구 마모도, 장비 유형 정보
- 상태 정보: 고장 여부 및 고장 유형 분류

✅ 제조 데이터 학습의 장점

- 실제 현장과 유사한 데이터 패턴
- 다양한 변수 간 상관관계 학습 가능
- 데이터 전처리 실습에 적합한 복잡성



제조 데이터셋을 볼 때 먼저 확인할 것

한 행은 무엇을 의미하는가?

하나의 측정 시점? 하나의 제품? 하나의 배치?
데이터의 기본 단위를 명확히 파악해야 함

각 컬럼은 어떤 현장 정보를 담는가?

센서 데이터: 온도, 압력, 진동 등

운전 조건: 속도, 전류, 설정값 등

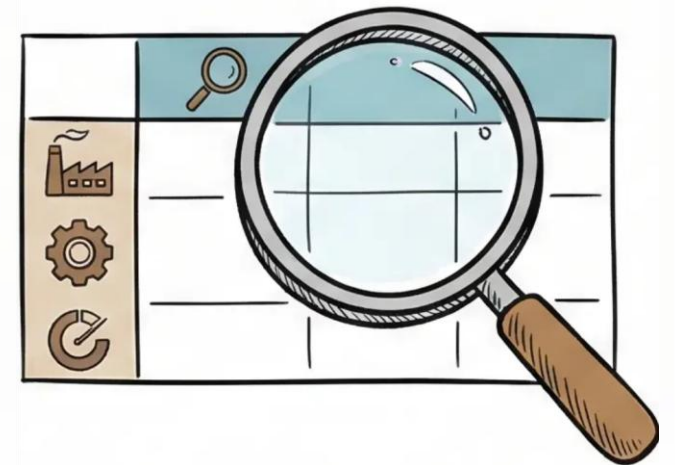
품질 지표: 불량률, 치수, 특성값 등

결과 변수와 설명 변수를 구분할 수 있는가?

결과 변수: 예측하고 싶은 목표 (고장, 품질 등)

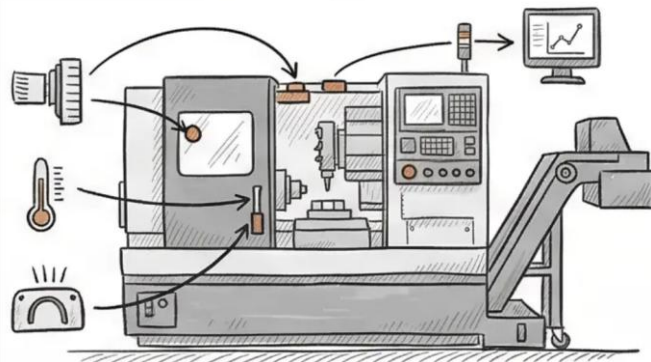
설명 변수: 결과에 영향을 주는 요인들

💡 데이터 해석의 출발점
“숫자를 보기 전에
의미를 먼저 이해하자”



주요 컬럼 한눈에 보기

AI4I 2020 데이터셋의 핵심 컬럼 분류



식별 정보

- UDI (Unique Device Identifier)
- Product ID
- Type (제품 유형)



센서 및 운전 변수

- Air temperature [$^{\circ}\text{C}$]
- Process temperature [$^{\circ}\text{C}$]
- Rotational speed [rpm]
- Torque [Nm]
- Tool wear [min]

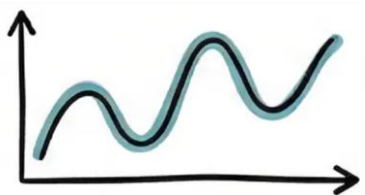


고장 상태 정보

- Machine failure
- TWF (Tool Wear Failure)
- HDF (Heat Dissipation Failure)
- PWF (Power Failure)
- OSF (Overstrain Failure)
- RNF (Random Failure)

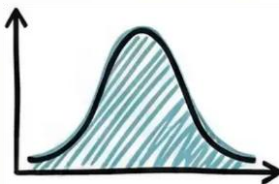
수치형 데이터

- 특징: 연속적인 측정값



- 예시: Air temperature [$^{\circ}\text{C}$], Rotational speed [rpm], Torque [Nm], Tool wear [min]

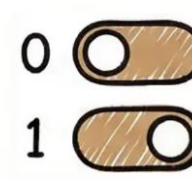
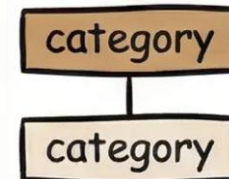
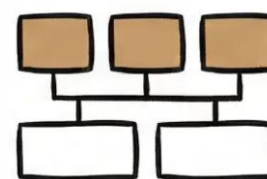
- 처리 방식: 평균, 분산, 분포 분석



처리 방식이 다르므로
먼저 나누어 봐야 한다

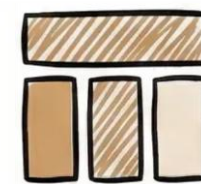
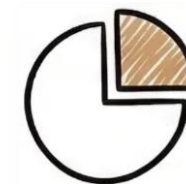
범주형 데이터

- 특징: 구분되는 카테고리



- 예시: Type (L, M, H), Machine failure (0, 1), Failure modes

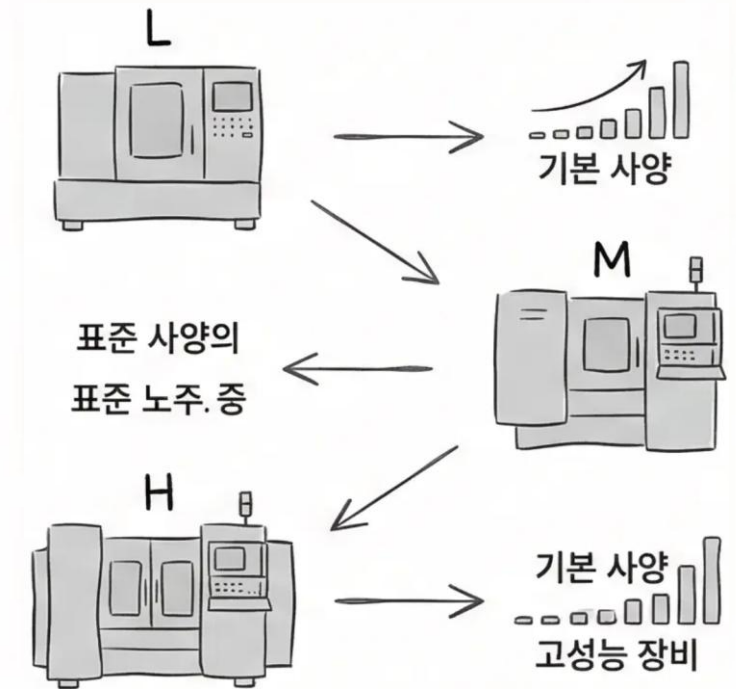
- 처리 방식: 빈도, 비율, 그룹 분석



장비 유형의 의미

- 단순 문자열이 아니라 중요한 해석 단서
- 유형별 그룹 비교에 자주 사용
- 제조 데이터에서는 범주형도 중요하다

L (Low): 기본 사양 장비
M (Medium): 표준 사양 장비
H (High): 고성능 장비



제조 현장에서 Type 컬럼이 말해주는 것

L (Low): 기본 사양 장비
M (Medium): 표준 사양 장비
H (High): 고성능 장비

EDA는 왜 먼저 해야 할까

- 데이터를 이해하지 않고 바로 분석하면 실수가 많아진다
 - 잘못된 가정으로 시작하는 분석 ⚠
 - 데이터 품질 문제를 놓치고 진행하는 위험 ⚠
- EDA는 그래프 그리기가 아니라 상태 점검이다
 - 결측치, 중복, 이상치 확인이 우선 ✅
 - 데이터의 실제 모습을 파악하는 과정
- 좋은 전처리는 좋은 EDA에서 시작된다
 - 문제를 미리 발견하고 대응 방향 설정
 - 분석 결과의 신뢰성 확보



제조 현장에서는 센서 오류, 기록 누락, 설비 교체 등으로 인한 데이터 변화가 자주 발생합니다.
EDA로 이런 상황을 미리 파악해야 정확한 분석이 가능합니다.

데이터 품질을 먼저 점검하자

- 결측치 (Missing Values)
 - 빈 값이나 누락된 데이터 확인
 - 분석 결과를 왜곡시킬 수 있음
- 중복 데이터 (Duplicates)
 - 동일한 기록이 여러 번 입력된 경우
 - 통계와 집계 결과에 영향
- 자료형 적절성 (Data Types)
 - 숫자는 숫자형으로, 문자는 문자형으로
 - 잘못된 타입은 계산 오류 발생
- 값의 범위 (Value Ranges)
 - 상식적으로 말이 되는 범위인지 확인
 - 온도가 음수 천도? 속도가 음수?



품질 점검이 먼저인 이유

불량 데이터로 분석하면 잘못된 결론에 도달합니다. 시간을 들여서라도 데이터 상태를 먼저 파악하는 것이 중요합니다.

결측치 확인하기

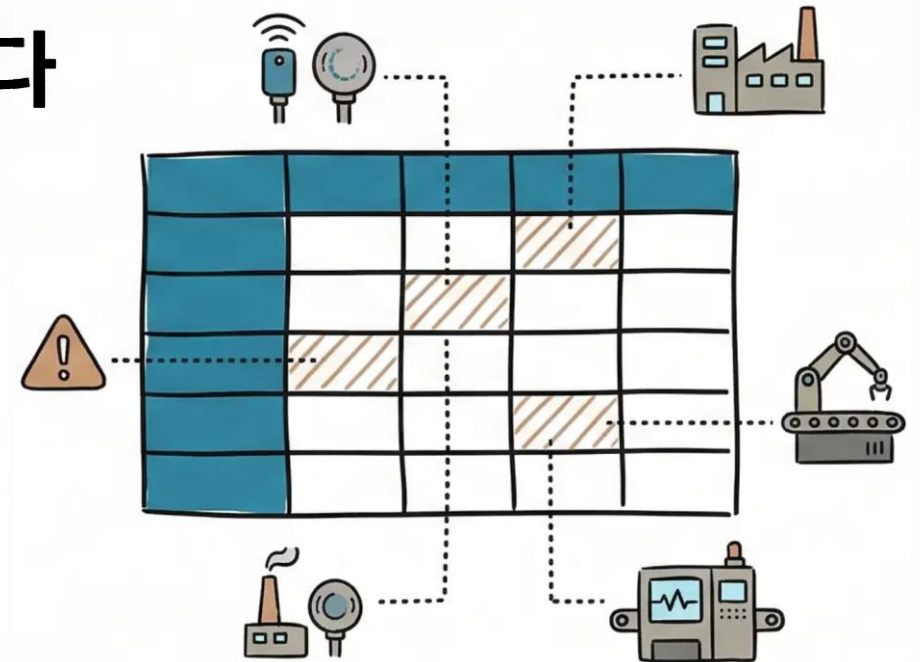
- 결측치가 있는지 먼저 확인한다
- 없더라도 확인하는 습관이 중요하다 
- 결측은 처리 전에 의미를 먼저 생각해야 한다 

```
```python
결측치 확인하는 기본 방법
df.isnull().sum()
df.info()
```
```



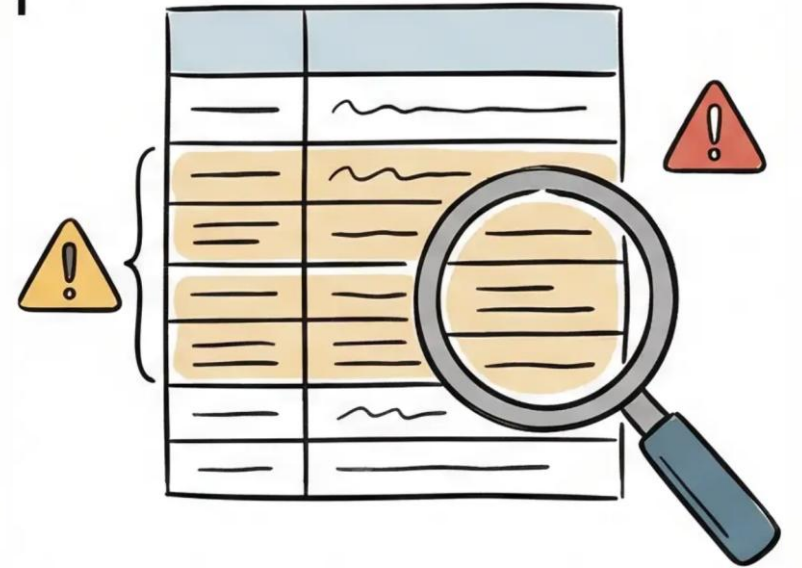
제조 현장에서 결측이 생기는 이유

- 센서 고장이나 통신 오류
- 교대 시간 데이터 누락
- 설비 점검 중 측정 중단
- 수동 입력 과정의 실수



중복 데이터 확인하기

- **중복은 건수와 평균을 왜곡합니다** ⚠
 - 동일한 데이터가 여러 번 계산되어 통계가 부정확해집니다.
 - 분석 결과의 신뢰성을 크게 떨어뜨립니다
- **적재 과정의 실수일 수도 있습니다** ✓
 - 파일을 여러 번 불러오는 과정에서 발생
 - 시스템 오류로 인한 중복 저장
- **분석 전 한 번은 꼭 점검해야 합니다** ✓
 - `df.duplicated().sum()`으로 중복 행 개수 확인
 - 전체 데이터 대비 중복 비율 파악

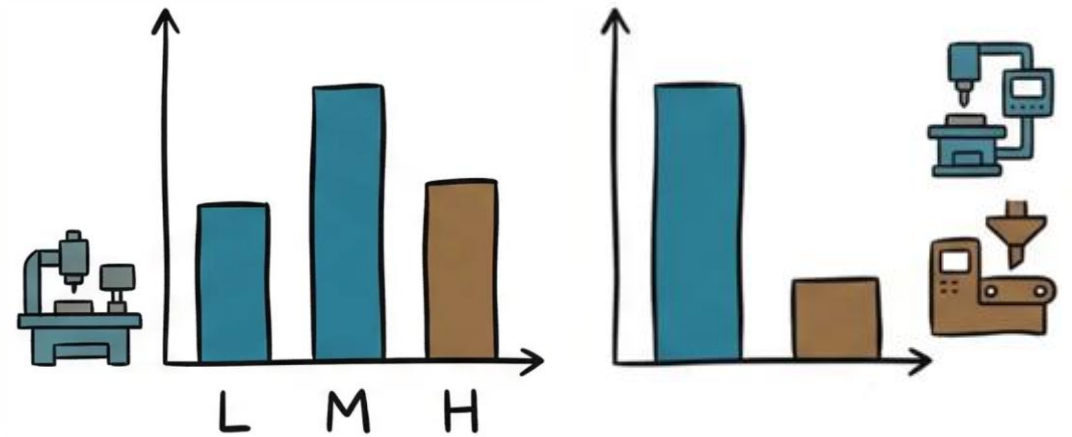


💡 실무 팁

중복 제거 전에 왜 중복이 발생했는지 원인을 먼저 파악하세요. 의미 있는 중복일 수도 있습니다.

범주형 데이터 분포 보기

- 장비 타입 분포 확인
- 고장 여부(Machine Failure) 분포 확인
- 범주형 분포는 현장 해석의 출발점
- 키분포는 현장 해석의 출발점
- Type별 데이터 건수가 균등한지 확인
- 고장 발생률이 어느 정도인지 파악
- 불균형한 분포는 분석 시 고려사항



왜 중요한가?

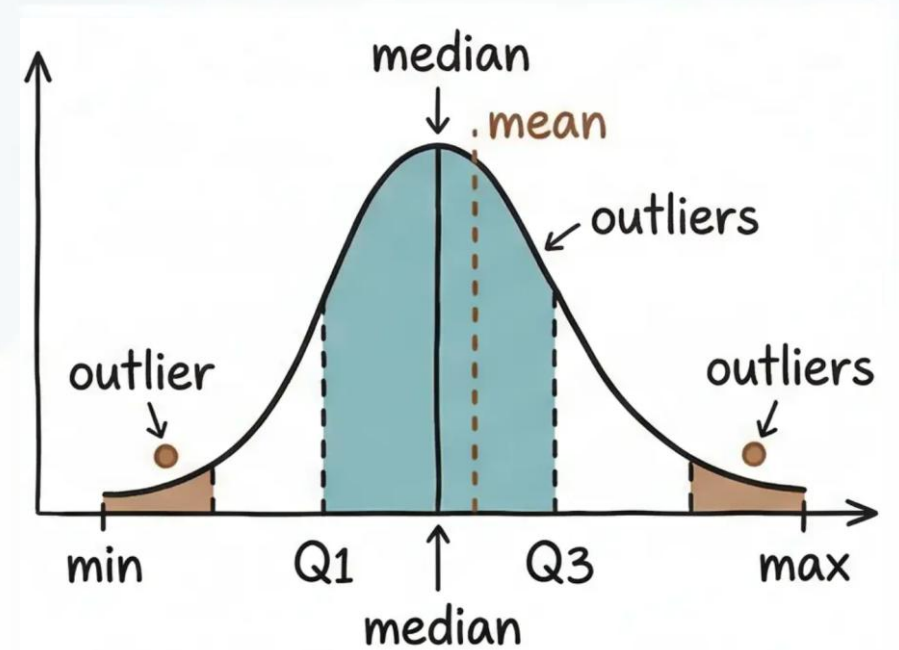
범주형 분포를 통해 현장의 실제 상황을 이해하고,
데이터 분석 방향을 설정할 수 있습니다

수치형 데이터 분포 보기

- 핵심 개념: 평균만 보면 데이터의 실제 모양을 놓치기 쉽습니다

- 확인해야 할 요소들:

-  • 최솟값과 최댓값으로 데이터 범위 파악
-  • 사분위수(Q1, Q3)로 데이터 집중도 확인
-  • 중앙값과 평균의 차이로 치우침 정도 판단
-  • 이상치 존재 여부와 위치 확인

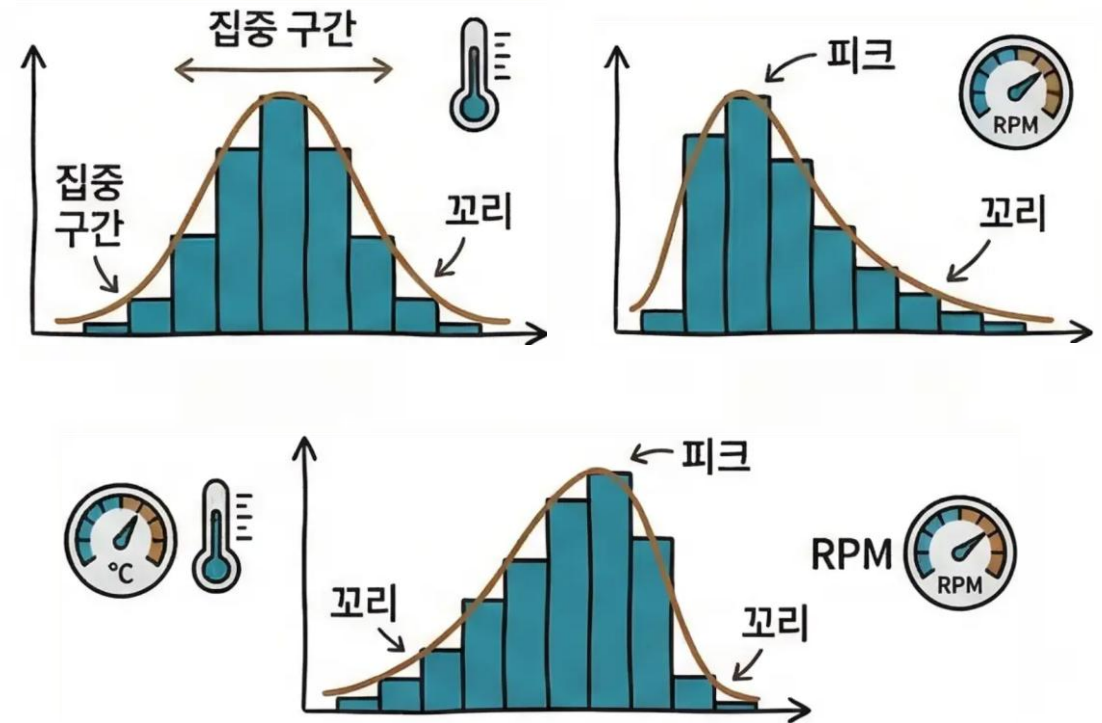


제조 데이터에서 주의할 점

- 센서 값의 급격한 변화는 장비 이상 신호일 수 있음
- 정상 범위를 벗어난 값도 의미 있는 정보

히스토그램으로 분포 이해하기

- 분포의 모양 파악 - 데이터가 어느
- 구간에 집중되어 있는지 한눈에 확인
- 치우침 정도 확인 - 좌측/우측으로
치우친 분포인지 대칭적인지 판단
- 이상치 영역 발견 - 극값에서 떨어진
구간의 데이터 존재 여부 확인
- 제조 데이터 해석 - 온도, 회전수, 토크
등의 등의 정상 범위와 패턴 이해

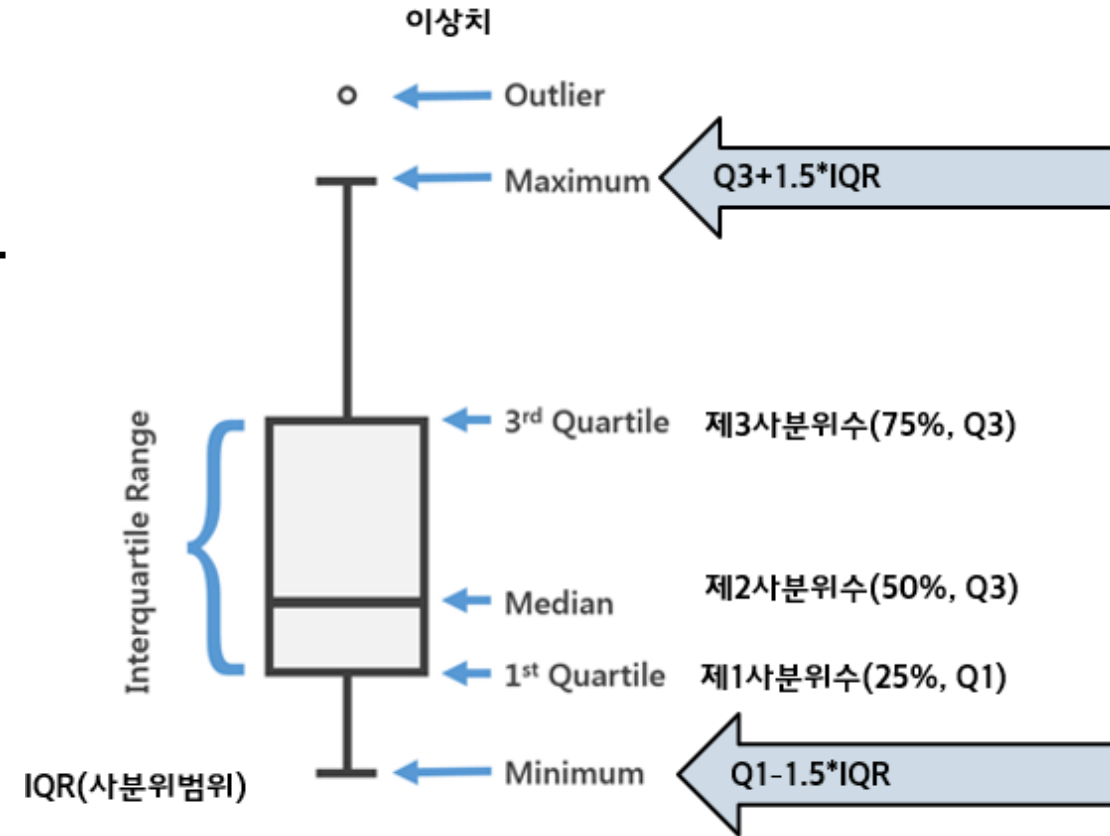


💡 실무 팁

평균값만으로는 놓치기 쉬운 데이터의 실제 분포 특성을 히스토그램으로 명확하게 파악할 수 있습니다

박스플롯으로 이상치 보기

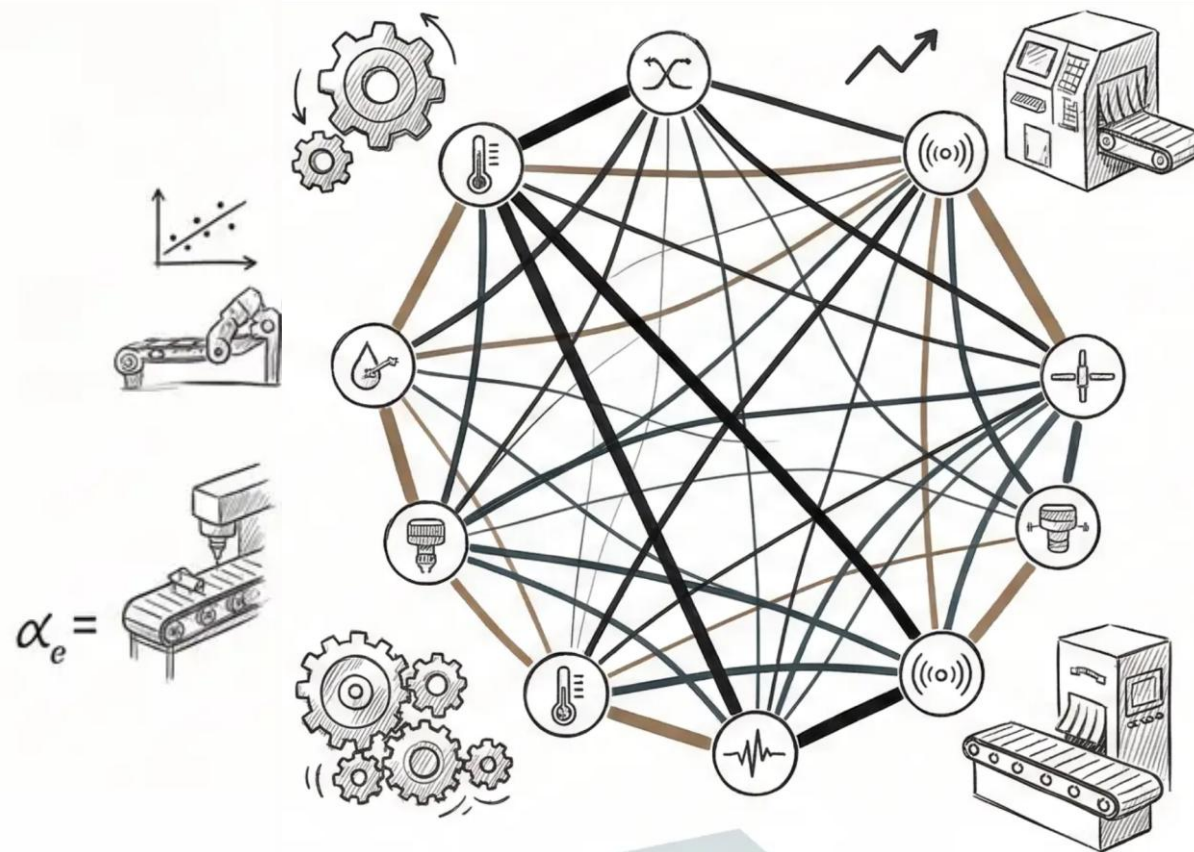
- 분포와 이상치를 동시에 확인
 - 데이터의 중앙값, 사분위수, 범위를 한눈에 파악
 - 상자 밖의 점들이 이상치 후보
- 제조 현장에서 이상치의 의미
 - 단순한 오류가 아닐 수 있음
 - 설비 이상 신호의 가능성
 - 공정 변화의 조기 경보
- 박스플롯 해석 포인트
 - 상자: 25%~75% 구간 (정상 범위)
 - 수염: 일반적인 변동 범위
 - 점: 주의 깊게 살펴볼 값들



제조 데이터의 이상치는
삭제하기 전에 먼저
현장 상황을 확인하세요

상관관계를 왜 확인할까

- 변수들이 함께 움직이는지 확인
 - 온도가 올라가면 품질이 떨어지는가?
 - 회전수와 진동은 연관이 있는가?
- 중복된 정보가 있는지 힌트 획득
 - 비슷한 패턴을 보이는 센서들 발견
 - 불필요한 변수 제거 가능성 파악
- 분석 방향성 설정
 - 어떤 변수를 집중해서 볼지 결정
 - 예상치 못한 관계 발견의 기회



중요한 주의사항

상관관계 $\neq$ 인과관계. 높은 상관관계가 있어도 원인-결과를 의미하지 않습니다

제조 데이터 EDA에서 자주 하는 실수

평균만 보고 판단하기 ⚠

분포의 모양과 편차를 놓치기 쉽습니다

이상치를 무조건 제거하기 ⚠

제조 현장에서는 중요한 이상 신호일 수 있습니다

범주형 컬럼을 무시하기 ⚠

장비 유형별 차이는 핵심 인사이트입니다

맥락 없이 그래프만 해석하기 ⚠

현장 지식 없는 분석은 잘못된 결론을 낳습니다



제조 데이터 분석의 원칙

“통계보다 현장 맥락을 먼저 이해하고, 이상치도 의미 있는 신호로 접근하세요”

컬럼명 표준화하기

Problem Section:

원본 데이터의 컬럼명은 일관성이 없습니다
공백, 대괄호, 특수문자가 섞여 있어 처리가 어렵습니다
한글과 영어가 혼재되어 있습니다

```
df.rename(columns={
    'Air temperature [°C]': 'air_temp_c',
    inplace=True)
```

Solution Section:

공백을 언더스코어로 변경
특수문자 제거 또는 통일
소문자로 일관성 유지
의미 있는 축약어 사용

Air temperature [°C]
Process temperature [°C]
temperature
...

표준화 →

air_temp_c
process_temp_c

표준화된 컬럼명은 이후 merge 작업과 SQLite 적재 시 오류를 방지하고 코드 가독성을 크게 향상시킵니다

자료형을 바르게 맞추기

• 숫자는 숫자로

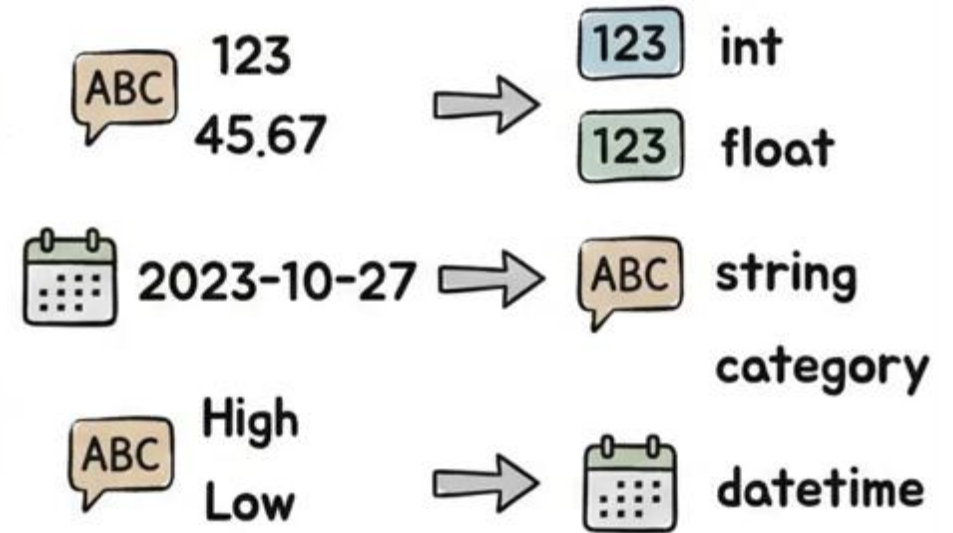
- 문자열로 저장된 수치 데이터를 numeric 타입으로 변환
- 계산과 집계 가능한 상태로 만들기

• 문자는 문자로

- 범주형 데이터를 string 또는 category 타입으로 설정
- 메모리 효율성과 처리 속도 향상

• 날짜는 날짜로

- 날짜 문자열을 datetime 타입으로 변환
- 시계열 분석과 날짜 연산 가능



“자료형이 맞아야 계산과 비교가 가능합니다

- `pandas.to_numeric()`
- `pandas.to_datetime()`
- `astype()` 메소드 활용

”

문자열 데이터 정리하기

불필요한 공백 제거와 표기 통일을 통해 코드형 문자열의 데이터 정합성을 높이는 방법을 실습합니다

• 주요 정리 작업

- 공백 제거: 앞뒤 여백과 중간 불필요한 공백
- 표기 통일: 대소문자, 구분자, 약어 표준화
- 특수문자 정리: 일관되지 않은 기호 제거
- 코드 규칙: 장비코드, 부품번호 형식 맞추기

• 실무 예시

- " L-001 " → "L-001"
- "medium", "MEDIUM" → "Medium"
- "Type_A", "Type-A" → "Type_A"
- "설비01", "설비 01" → "설비01"
- Before/After 비교로 정리 효과 확인



범주형 값을 정리하는 방법

•같은 의미, 다른 표기 통합

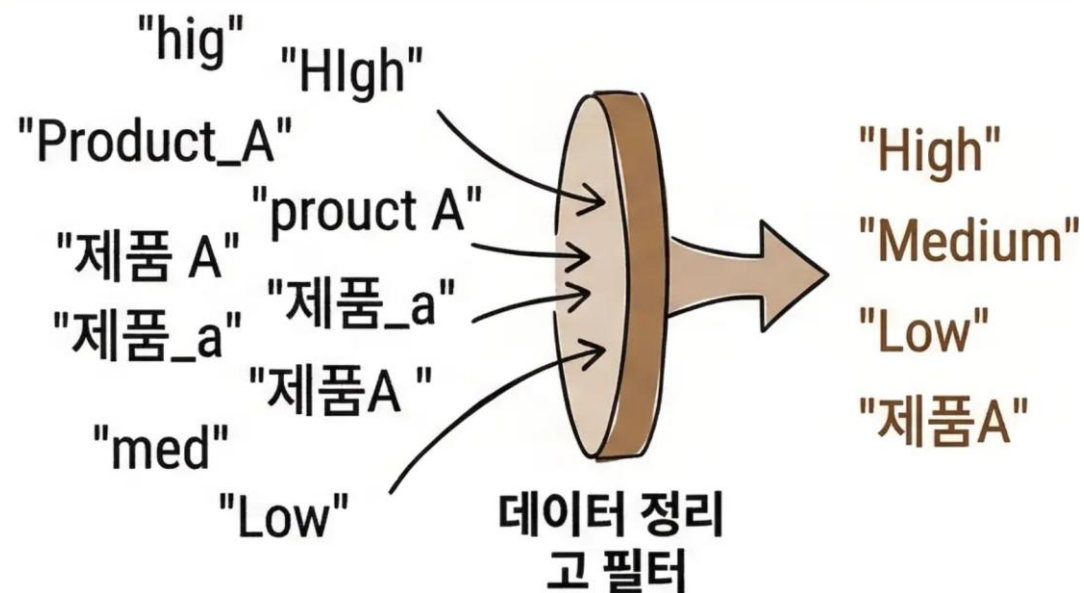
- "High","HIGH","high" → "High"
- "제품A","제품 A","제품_A" → "제품A"

•오타와 표기 차이 수정

- 공백 제거 및 통일
- 대소문자 표준화
- 특수문자 정리

•그룹 분석 준비

- 일관된 값으로 통합 후 집계 가능
- 정확한 빈도 계산 실현
- 시각화와 보고서 품질 향상



실무 팁

범주형 정리는 분석 전 필수 단계입니다.
정리하지 않으면 같은 의미가 여러 그룹으로
나뉘어 잘못된 결과를 만들 수 있습니다.

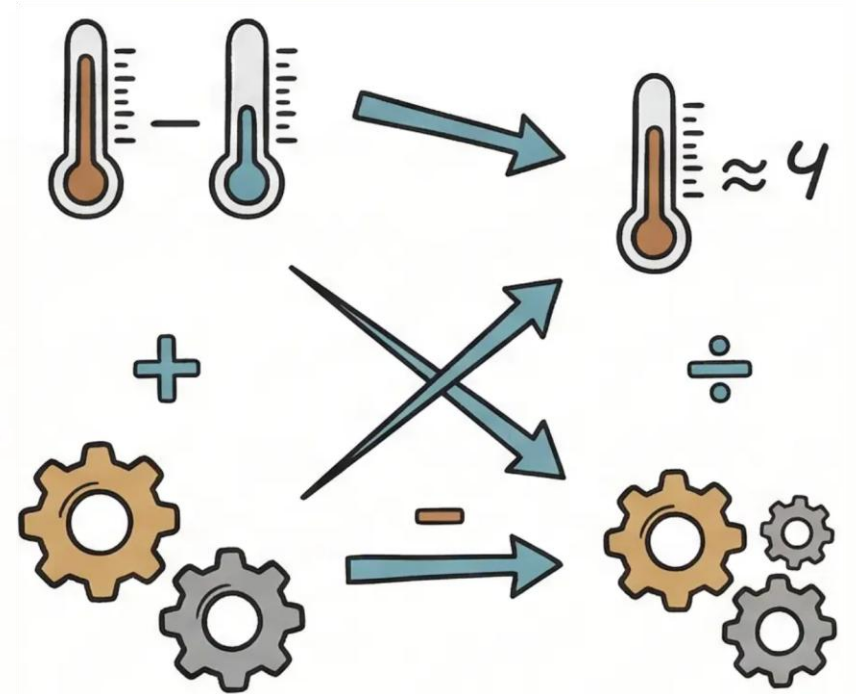
파생 변수 만들기

■ 기존 컬럼 조합의 힘

- 온도 차이 = 공정 온도 - 대기 온도
- 마모 구간 = 공구 마모 시간을 단계별로 분류
- 회전 효율 = 실제 RPM / 목표 RPM

■ 현장 해석 가능한 변수

- 단순 계산보다 의미 있는 지표 생성
- 도메인 지식을 반영한 변수 설계
- 분석과 모델링에 직접 활용 가능



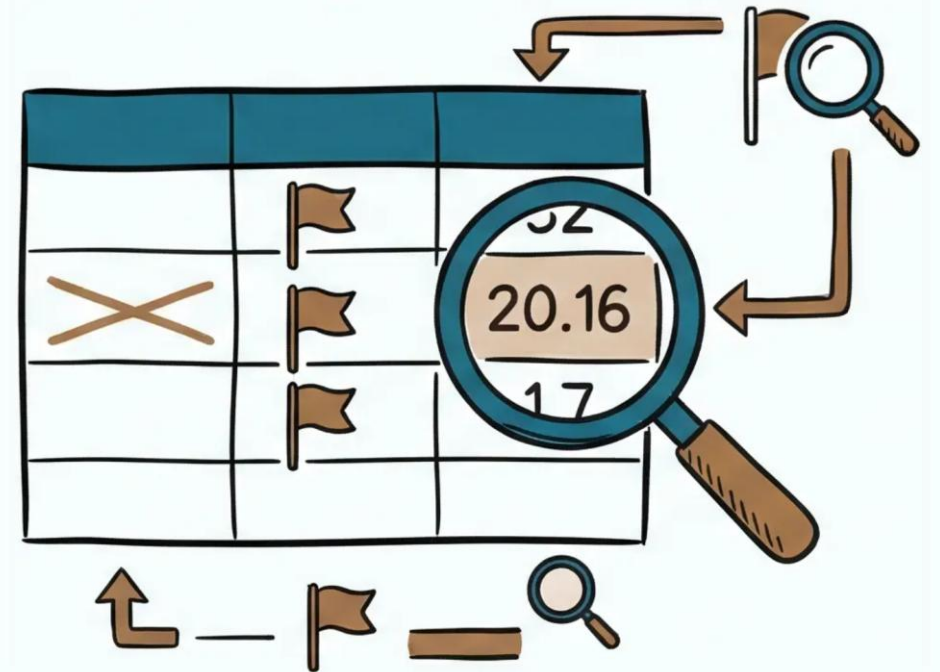
파생 변수는 데이터의 숨겨진 패턴을 드러내는 열쇠입니다.
제조 현장의 경험과 Python 코드가 만나는 지점이죠.

이상치 플래그 컬럼 추가하기

- 핵심 원칙: 이상치를 바로 삭제하지 않습니다
- 플래그 접근법: 별도 컬럼으로 이상치 여부를 표시합니다
- 실무 관점: 나중에 판단할 수 있는 여지를 남겨둡니다

실무에서 중요한 이유

- 추적 가능성 유지
- 판단 근거 보존
- 도메인 전문가 검토 가능
- 데이터 손실 방지



조건 필터링과 정렬

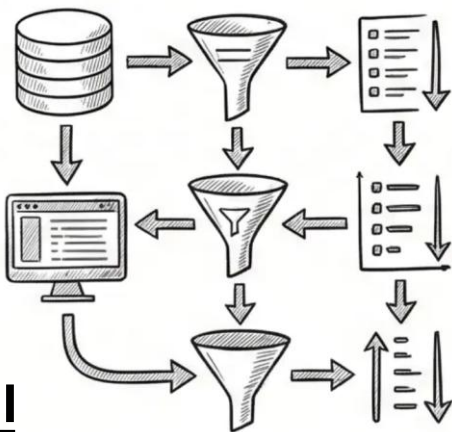
효율적인 데이터 점검과 보고를 위한 핵심 기술

조건 필터링의 핵심

- 특정 조건을 만족하는 데이터만 추출
- 고장 발생 데이터만 따로 분석
- 특정 장비 타임별 성능 비교
- 임계값을 벗어난 이상 데이터 식별

정렬의 실무 활용

- 우선순위가 높은 데이터를 먼저 확인
- 시간순, 중요도순 데이터 배열
- 상위 N개 문제 상황 파악
- 트렌드와 패턴 발견



실무 예시

고장 발생 데이터만 필터링

```
failure_data = df[df['Machine failure'] == 1]
```

```
sorted_data = df.sort_values('Tool wear [min]',  
                             ascending=False)
```

복합 조건으로 위험 상황 식별

```
risk_data = df[(df['Tool wear [min]'] > 200) &  
               (df['Rotational speed [rpm]'] < 1000)]
```

그룹화와 집계로 요약하기

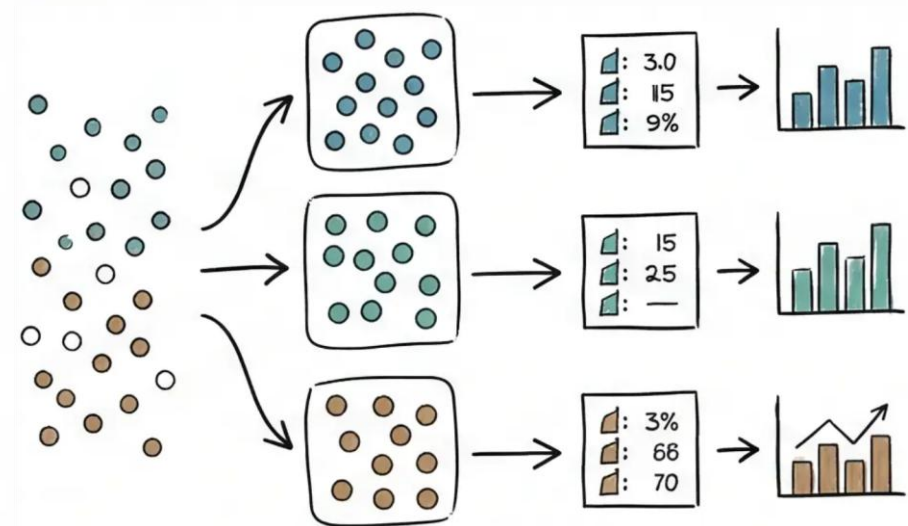
주요 그룹화 시나리오

- 장비 타입별 평균 성능 분석
- 고장 여부별 운전 조건 비교
- 시간대별 생산량 요약

핵심 Pandas 함수

- 'groupby()': 그룹 기준 설정
- 'mean()', 'count()', 'sum()': 집계 함수
- 'agg()': 여러 집계를 한번에

`df.groupby('Type').mean()`



Data Grouping and Aggregation Process

실무 활용 예시

- 일일 보고서 자동 생성
- 장비별 성능 벤치마킹
- 이상 패턴 조기 발견

실무 팁

그룹별 요약은 제조 현장 보고서의 90%를 차지합니다.
한 번 익히면 매일 써먹는 핵심 스킬입니다.

전처리 결과 최종 확인

검증해야 할 핵심 요소들

최종 컬럼 목록과 데이터 타입 확인

전체 데이터 크기 (행/열 수) 점검

샘플 데이터로 변환 결과 검토

컬럼 검증: 모든 파생 변수와 정제된 컬럼이 포함되었는지 확인

데이터 무결성: 원본 대비 적절한 행 수 유지 여부 점검

품질 확인: 이상치 플래그와 정제된 값들이 올바르게 적용되었는지 검토

전처리 완료 체크리스트

- ✓ df.shape로 최종 크기 확인
- ✓ df.columns로 컬럼 목록 점검
- ✓ df.head()로 샘플 데이터 검토

- ✓ df.info()로 자료형 최종 확인



정제 결과 저장하기

• 저장은 실습의 끝이 아니라 다음 단계의 시작

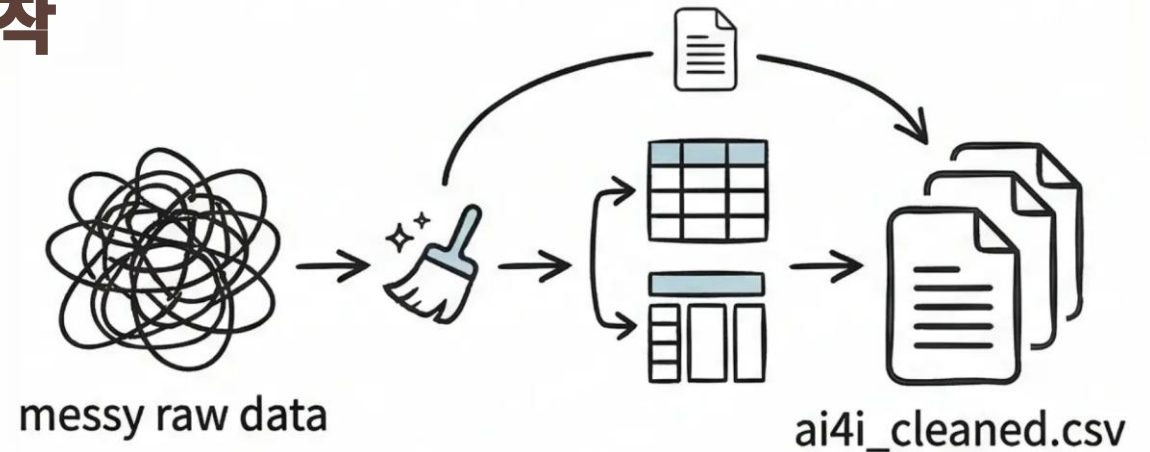
- 정제된 데이터를 파일로 보존
- 재사용 가능한 형태로 저장
- 다음 단계 실습을 위한 준비

• 올바른 저장 방법

- 명확한 파일명 사용 (ai4i_cleaned.csv)
- 적절한 인코딩 설정 (UTF-8)
- 저장 위치와 경로 확인

• 저장 후 검증 습관

- 파일이 정상적으로 생성되었는지 확인
- 데이터 크기와 구조 재점검
- 다시 읽어서 내용 확인



💡 실무 팁

저장만 하고 끝내지 마세요. 저장된 파일을 다시 읽어 확인하는 습관이 3주차 데이터베이스 적재 검증과도 연결됩니다.

저장 후 다시 읽어 확인하기

■ 저장만 하고 끝내지 않는 습관

- 파일을 저장한 후 반드시 다시 읽어서 확인
- 저장 과정에서 발생할 수 있는 오류 조기 발견

■ 검증해야 할 주요 항목들

- 행과 열의 개수가 원본과 동일한지 확인
- 주요 컬럼의 데이터 타입 유지 여부
- 한글 인코딩이 정상적으로 저장되었는지 점검

■ 실무에서의 중요성

- 다음 단계 작업의 신뢰성 확보
- 데이터 손실 방지를 위한 필수 과정



💡 검증 코드 예시

```
# 저장 후 재확인
df_reloaded = pd.read_csv('ai4i_cleaned.csv')
print(f"원본: {df_original.shape}")
print(f"재로드: {df_reloaded.shape}")
```

1주차 실습 흐름 정리

데이터 처리의 전체 여정을 한눈에 보기



1단계: 원본 데이터 읽기

- AI4I 2020 데이터셋 로드
- 기본 구조 파악

2단계: 데이터 상태 점검

- shape, info, describe
describe 활용
- 결측치와 중복 확인

3단계: 기초 EDA

- 분포 시각화
- 상관관계 분석
- 이상치 탐지

4단계: 전처리

- 컬럼명 표준화
- 파생변수 생성
- 이상치 플래그 추가

5단계: 저장

- ai4i_cleaned.csv 생성
- 저장 후 재확인

1주차 결과 확인

데이터 구조 이해와 파생 변수 생성, 정제 파일 저장 등 1주차 학습 성과를 최종 확인합니다

• 우리가 달성한 것들

• 데이터 구조를 완전히 이해했습니다

- AI4I 2020 데이터셋의 행과 열 의미 파악
- 수치형과 범주형 컬럼 구분
- 제조 현장 데이터의 특성 이해 √√√

• 실무형 EDA를 수행했습니다

- 결측치와 중복 데이터 점검
- 분포와 이상치 확인
- 상관관계 분석

• 전처리 기술을 적용했습니다

- 컬럼명 표준화
- 파생 변수 생성
- 이상치 플래그 추가



 **핵심 결과물**

ai4i_cleaned.csv

**다음 주 통합 작업을 위한
정제된 데이터 완성**

1주차 요약

- 읽기보다 확인이 중요하다

데이터를 불러오는 것은 시작일 뿐, 상태를 점검하고 이해하는 것이 핵심



- EDA는 상태 점검이다

그래프 그리기가 목적이 아니라 데이터 품질과 구조를 파악하는 과정



- 전처리 결과는 반드시 저장해야 한다

정제한 데이터를 파일로 저장해야 다음 단계로 이어갈 수 있음



다음 주 예고

• 2주차 주제: 다양한 데이터 소스 읽기와 통합

• 핵심 활동:

- 기준정보 CSV와 여러 파일 결합
- JSON 형태 데이터 처리
- API 데이터 개념 학습
- merge 기능으로 데이터 통합
- API 데이터 개념 •

- 최종 결과물: ai4i_enriched.csv
- 통합 내용: 1주차 정제 데이터 + 외부 소스 정보
- 실습 방향: 실제 제조 현장의 다양한 데이터 소스 경험

